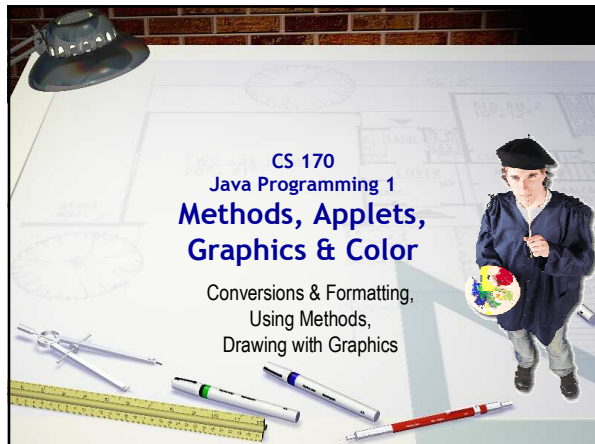
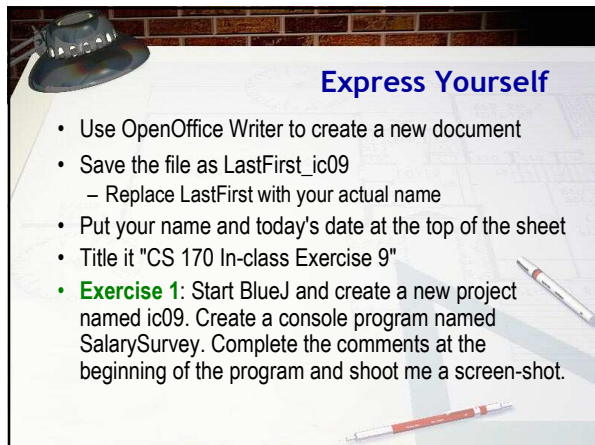
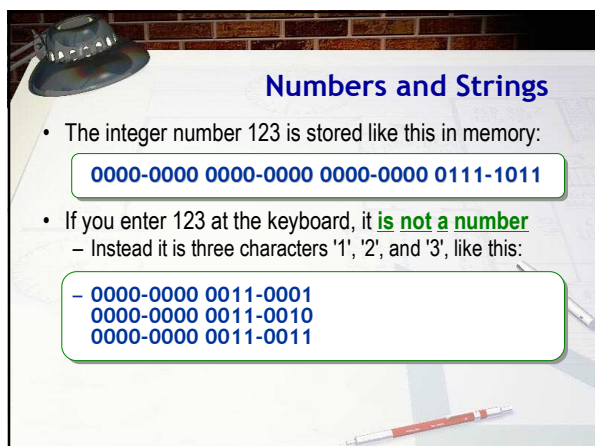








Strings to Numbers

- What do you do when you **can't use** a **Scanner**?
- For calculations, text must be converted to numbers
 - **Integer** class has a **parseInt()** method to do that
 - This is a **static** method, so you use the class name
 - Getting numeric input from a **TextField** in a GUI app

```
String aStr = someTF.getText();  
int num = Integer.parseInt(aStr);
```

Express Yourself

- **Exercise 2:** Use Code Pad to create the following String variables initialized as shown:
 - `String one = "123";`
 - `String two = " 456";` // note space before
 - `String three = "789 ";` // note space after
 - `String four = "9.5";`
 - `String five = "five";`
- Use **Integer.parseInt()** to convert each number. Shoot me a screen-shot of the Code Pad window with the results.

Other Conversions

- Java has several "wrapper" classes like **Integer**
 - Discussed briefly on page 158-59 in your textbook
- For floating-point input, use the **Double** class
 - You can use either of these ways to convert

```
String dStr = dTF.getText(); // String from user  
double a = Double.parseDouble(dStr);  
double b = Double.valueOf(dStr).doubleValue();
```

- **Exercise 3:** Convert your variables to doubles. Shoot me a screen-shot.

From Numbers to Strings

- Once you've completed your calculations, you'll want to display them on the screen
 - You **cannot** display "raw" binary numbers
 - Instead, you have to turn the results **back into a String**
- Concatenation **automatically** turns numbers into strings

```
double result = a / b + c; // some calculation
String output = "result = " + result;
System.out.println(output);
```

Beginning mileage	149254.7
Current mileage	149375.3
Cost of fill-up	7.54
Gallons purchased	3.52
Miles per gallon	34.36176767676768
Cost per mile	0.2223072964491505

- Problem is you don't have control over the **format**

printf() and String.format()

- The C standard library introduced the **printf()** method
 - Java 5 adds **printf()** to the **System.out** object
 - Very concise, but a little "unsafe"; be careful when using
- System.out.printf("Result: %10.2f", var);**
 - First parameter is a **format string** which can contain:
 - Normal text which will print as is ("Result: ")
 - Placeholders or **format specifiers** for each variable
 - Format specifiers start with **%**, followed by format **type**
 - %f** for floats and double; **%d** for integers; **%s** for string

```
2004: Java
System.out.printf("%10.2f", x);
```

More on Formatting

- Can include width, precision, flags with specifier
 - "%5d"** – integer right-aligned in a 5 character field
 - "%5.2f"** – real with 2 digits of precision, 5-wide
 - "%-20s"** – string left aligned in a field 20 wide
 - "%,d"** – integer with decimal separator
- You must provide a variable for **each** specifier
 - The variable must be the correct type
 - Not checked by the compiler

Express Yourself

- **Exercise 4:** Use Code Pad to create the following variables initialized as shown:
 - `String six = "Midterm Score";`
 - `int seven = 456;`
 - `double eight = 9876543212345.678;`
- Use `String.format()` to format each number.
 - Format six in a field 20 wide, left-aligned, delimited by |
 - Format six in a field 5 wide, right-aligned, delimited by |
 - Format seven in a field 6 wide
 - Format eight with 5 decimals, a dollar-sign & commas

Express Yourself

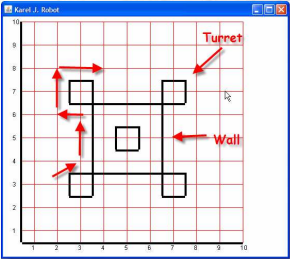
- **Exercise 5:** Complete the SalarySurvey application:
 - Ask the user to enter their first name and desired yearly salary, separated by a space.
 - Use the `Scanner`'s `next()` method to read into two `String` variables.
 - Use the conversion methods to convert the salary into a double and store it in a double variable.
 - Use `printf()` to display the result like this:
- Name: Steve
- Desired salary: \$ 1,275,315.75
- Paste your code into ic09 along with a run of the program.

A Visit From Karel

- Methods can be used for more than just setting values or returning information about an object.
 - Provide a powerful way to solve a problem
 - Put multiple steps into a single, simple method
 - Functional decomposition or divide and conquer
- Your last Karel problem was fairly long, a lot of steps
 - Let's use functional decomposition to complete a more complex job, in a simpler way
- **Exercise 6:** copy PatrolTheCastle project from Q: and open it in BlueJ. Create an object and call `task()`.

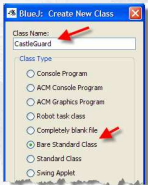
Karel Patrols the Castle

- Here's the castle and the route we want Karel to follow
- Think how many lines you'd have to write to patrol once around
 - Hard to understand
 - Hard to find your mistakes
- What if we had a robot that already knew how to patrol a castle?



Our First CastleGuard

- Inheritance allows us to create a new specialized version of our **OCRobot** and add new commands
- Exercise 7:** add a new Bare Standard Class to your project, and name it **CastleGuard**. Then, open the file in your editor.



Setting Up CastleGuard

- Add these changes to **CastleGuard** and compile:
 - `import cs170.*;` on line 1
 - `extends OCRobot` on the header line
 - Add a comment and your name at the top of the class



Setting Up the Task

- Add these changes to **PatrolTheCastle**
 - Change the **OCRobot** variable to a **CastleGuard**
 - Ignore the "don't modify this" comment
 - Initialize karel using the **CastleGuard()** constructor

```
11 public class PatrolTheCastle implements R
12 {
13     // Instance/static variables .....
14     private CastleGuard karel = null; //
15     // add your instance or static variables
16
17     public void task()
18     {
19         World.startFromURL(
20             "http://csjava.occ.occ.edu/~gilbert/
21             karel = new CastleGuard();
22     }
```

- **Exercise 8:** create a **PatrolTheCastle**, call **task()**

Adding turnRight

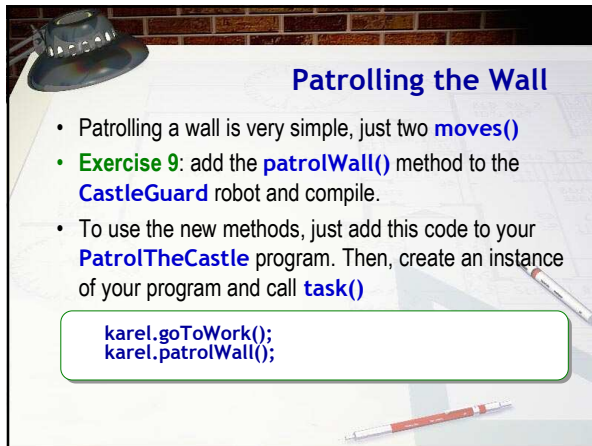
- The **OCRobot** can't **turnRight()** so you had a bunch of **turnLeft()** commands in your last homework.
- To add **turnRight()** to **CastleGuard**, add a method

```
public void turnRight()
{
    turnLeft();
    turnLeft();
    turnLeft();
}
```

Moving to the Start

- We can do the same thing to move Karel into the starting position, even though we'll only do it once

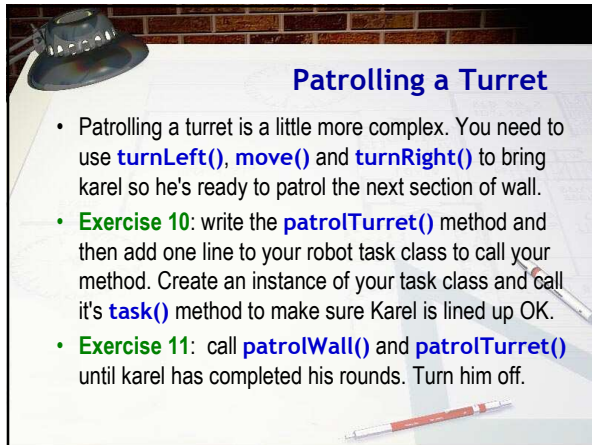
```
public void goToWork()
{
    move();
    move();
    move();
    turnRight();
    move();
    move();
    turnLeft();
}
```

Patrolling the Wall

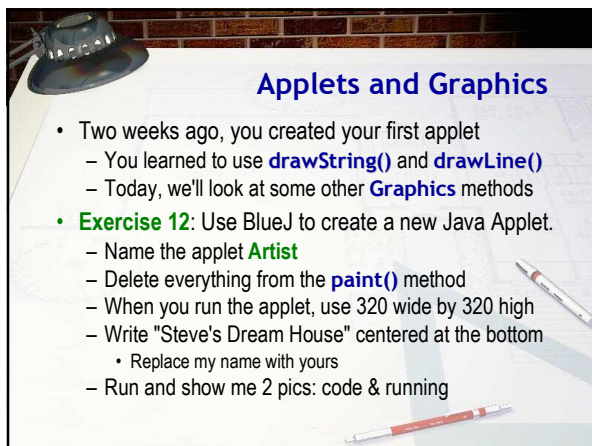
- Patrolling a wall is very simple, just two `moves()`
- **Exercise 9:** add the `patrolWall()` method to the `CastleGuard` robot and compile.
- To use the new methods, just add this code to your `PatrolTheCastle` program. Then, create an instance of your program and call `task()`

```
karel.goToWork();  
karel.patrolWall();
```



Patrolling a Turret

- Patrolling a turret is a little more complex. You need to use `turnLeft()`, `move()` and `turnRight()` to bring karel so he's ready to patrol the next section of wall.
- **Exercise 10:** write the `patrolTurret()` method and then add one line to your robot task class to call your method. Create an instance of your task class and call it's `task()` method to make sure Karel is lined up OK.
- **Exercise 11:** call `patrolWall()` and `patrolTurret()` until karel has completed his rounds. Turn him off.

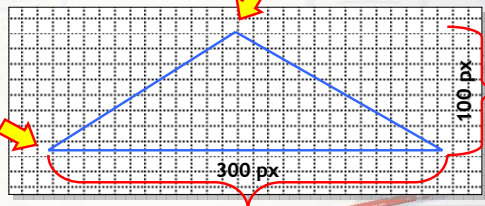


Applets and Graphics

- Two weeks ago, you created your first applet
 - You learned to use `drawString()` and `drawLine()`
 - Today, we'll look at some other `Graphics` methods
- **Exercise 12:** Use BlueJ to create a new Java Applet.
 - Name the applet **Artist**
 - Delete everything from the `paint()` method
 - When you run the applet, use 320 wide by 320 high
 - Write "Steve's Dream House" centered at the bottom
 - Replace my name with yours
 - Run and show me 2 pics: code & running

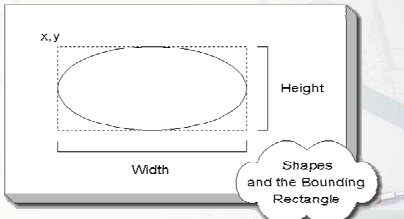
Hands On: Using drawLine()

- **Exercise 13:** Draw the roof using `drawLine()`
- Draw a triangle 100 pixels high, with a base of 300
- Center the apex over the base
 - Put the triangle 10 from the left, 10 down from the top



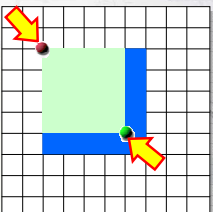
Drawing Closed Shapes

- Could use same 4 params to draw rectangles or ovals
 - **Instead**, Java's shapes use a "bounding box"
 - Uses **width** and **height** instead of x2, y2



Drawn versus Filled Shapes

- Coordinates used for x,y represent "points"
 - Infinitely small location located between pixels
 - Example: points 2,2 and 6,6
- Shape-drawing methods come in two varieties
 - Those that **draw** an outline
 - Use 1-pixel wide pen that hangs down/right
 - Those that **fill** an area
- Drawn shapes are 1-pixel bigger than filled shapes



Plain Rectangles

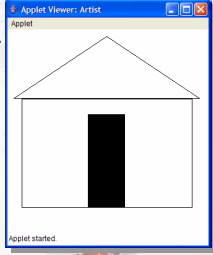
- Three methods:

```
void drawRect(int x, int y, int width, int height)
void fillRect(int x, int y, int width, int height)
void clearRect(int x, int y, int width, int height)
```

 - First draws outline anchored at x,y in foreground color
 - Will be one pixel higher and wider than requested
 - Second draws solid square in foreground color
 - Last works like `fillRect()`, using background color

Hands On: Using Rectangles

- Let's put this to work by drawing a "house" underneath the "roof" we've already created
- Paste code, screenshot running.
- Exercise 14:** Use `drawRect()` to paint a rectangle 275 pixels wide and 175 pixels high under the roof
- Exercise 15:** Use `fillRect()` to paint a door in the middle of the house



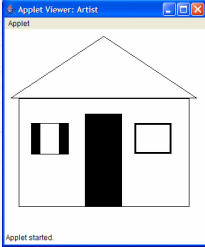
Drawing Thicker Lines

- Issue a **sequence** of `drawRect()` calls
 - Each time, increase `x`, `y` by 1
 - Decrease `width` and `height` by 2
- Use a combination of `fillRect()` and `clearRect()`

```
g.fillRect(10, 10, 100, 100);
g.clearRect(20, 20, 80, 80);
```

Hands On: Drawing Windows

- Let's put this to work by drawing some windows on the right and left side of the door.
- Paste code, screenshot running.
- Exercise 15:** What would you use to paint a window, like this?
- Exercise 16:** How about this?



Express Yourself

- Exercise 17:** Using what you've learned about rectangles and drawing thicker lines, look up the documentation for the **Graphics** class and see how you'd draw some circles.
 - Then, create another applet (in the same project), called **Target**, and draw an archery target like this. (Don't try to make it red for right now; we'll learn how to do that next.)
 - Shoot me a screen-shot of your program running and show me your code



The Color Class

- Simple RGB Class in the **java.awt** package
 - Don't confuse **java.awt.color package**
- Internal representation is a 32-bit (RGBA) integer
 - 8 bits red (R), 8 bits green (G), 8 bits blue (B)
 - Means we can have 256 levels of each color
 - 0 means "no color", 255 means "full color"
- You **don't care** about the internal representation
 - You'll use the public constructors and methods
 - Later, if the internal representation changes, your programs will still continue to work

Creating Color Objects

- Look up **Color** class, find constructor summary

```
Color(int red, int green, int blue);  
Color(float red, float, float blue);
```

- Pass **int** values between 0 (off) and 255 (on)
- Pass **float** values between 0F (off) and 1.0 (on)

- Here are some examples:

```
Color red = new Color(255, 0, 0);  
Color black = new Color(0, 0, 0);  
Color white = new Color(1.0F, 1.0F, 1.0F);  
Color lightGray = new Color(.9F, .9F, .9F);
```

The setColor() Method

- setColor()** changes the current painting color
- Use with your **Color** variable like this:

```
public void paint(Graphics g)  
{  
    Color brush = new Color(255, 128, 128);  
    g.setColor(brush);  
}
```

- Exercise 18:** Make your target red. Snap a picture.
- Exercise 19:** Put green windows and a red door on your artist's cottage. Show me your code, snap a pic.

Other Color Constructors

- Modern browsers can also display transparent colors

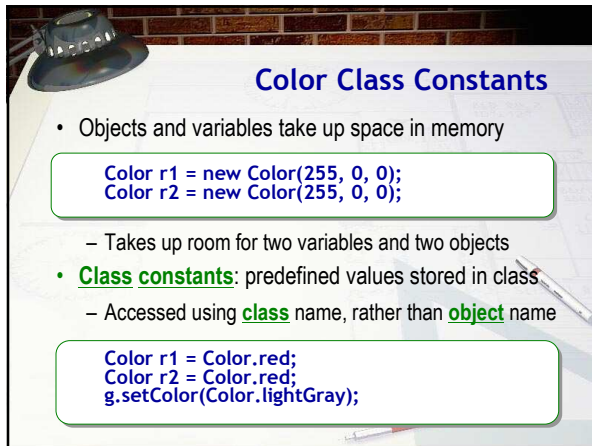
```
Color(int red, int green, int blue, int alpha);
```

- The alpha channel is 0 (transparent) to 255 (opaque)

- You can also specify colors using a single integer

```
Color(int colorValue);
```

- Mostly useful when working with HTML Hex colors

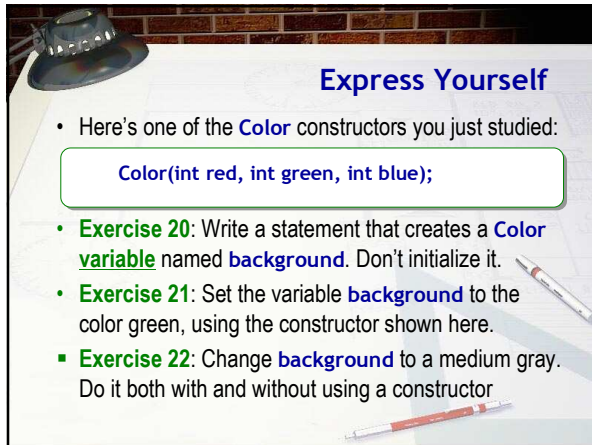


Color Class Constants

- Objects and variables take up space in memory
 - Takes up room for two variables and two objects
- **Class constants**: predefined values stored in class
 - Accessed using **class** name, rather than **object** name

```
Color r1 = new Color(255, 0, 0);  
Color r2 = new Color(255, 0, 0);
```

```
Color r1 = Color.red;  
Color r2 = Color.red;  
g.setColor(Color.lightGray);
```



Express Yourself

- Here's one of the **Color** constructors you just studied:

```
Color(int red, int green, int blue);
```
- **Exercise 20**: Write a statement that creates a **Color variable** named **background**. Don't initialize it.
- **Exercise 21**: Set the variable **background** to the color green, using the constructor shown here.
- **Exercise 22**: Change **background** to a medium gray. Do it both with and without using a constructor
